

HONEYPOT TRAP THE LIVING

Autonomous Adaptive Deception Environment (AADE)

Full Project Implementation Guide

SE IOT · 2025–2026

Team Members

Archita Roy

Phase 3 — Stealth Logging & Vsock

Python · Networking · Logging Systems

Sharvari Achrekar

Phase 2 — Gold Image & Human Noise

Linux Filesystems · Shell · Faker

Arnav Shirvadkar

Phase 1 & 4 — Virtualisation & Honeywall

Firecracker · KVM · iptables · Networking

Madhav Shukla

Phase 5 — Orchestrator & Dashboard

Python Flask · JavaScript · System Integration

Table of Contents

Section	Title	Page
1	Project Overview & Goals	3
2	Architecture Summary	4
3	Team Responsibilities Matrix	5
4	Phase 1 — Environment & Virtualisation (Arnav)	6
4.1	Pre-flight Checklist	6
4.2	Step-by-step Setup	7
4.3	GitHub Sources & Downloads	9
5	Phase 2 — Gold Image Creation (Sharvari)	11
5.1	Building the Fake Filesystem	11
5.2	Human-Noise Scripts	12
5.3	GitHub Sources & Downloads	14
6	Phase 3 — Stealth Vsock Logging (Archita)	16
6.1	Vsock Architecture	16
6.2	Python Listener Scripts	17
6.3	GitHub Sources & Downloads	19
7	Phase 4 — Honeywall & Containment (Arnav)	21
7.1	iptables Rules	21
7.2	Transparent Proxy Setup	23
8	Phase 5 — Orchestrator & Dashboard (Madhav)	25
8.1	Master Orchestrator Script	25
8.2	Flask Dashboard	27
8.3	GitHub Sources & Downloads	29
9	Integration & End-to-End Testing	31
10	Full Dependency & Install Reference	33
11	Risk Register & Mitigations	35
12	Project Timeline (Gantt)	37
13	Glossary	39

1. Project Overview & Goals

The Autonomous Adaptive Deception Environment (AADE) — nicknamed *The Living Trap* — is a next-generation honeypot system that replaces static, easily fingerprinted decoys with a self-adapting, AI-orchestrated environment. Where traditional honeypots present a frozen snapshot of a system, AADE presents a convincing, living illusion: files age, logs accumulate, CPU usage fluctuates, and even terminal responses are generated on-the-fly by an LLM so that even a seasoned attacker cannot distinguish the trap from a real workstation.

Core Design Principles

Principle	Description
Asymmetry Reversal	The defender now forces the attacker to expend resources inside a controlled maze.
Zero False Positives	Every alert fired from AADE corresponds to a real intrusion — no legitimate traffic enters.
Intelligence Extraction	Every command, payload, and lateral-movement attempt is captured out-of-band so the attacker cannot cover their tracks.
Modularity	Each phase (Virtualisation, Gold Image, Logging, Containment, Dashboard) is independently deployable.

Technology Summary

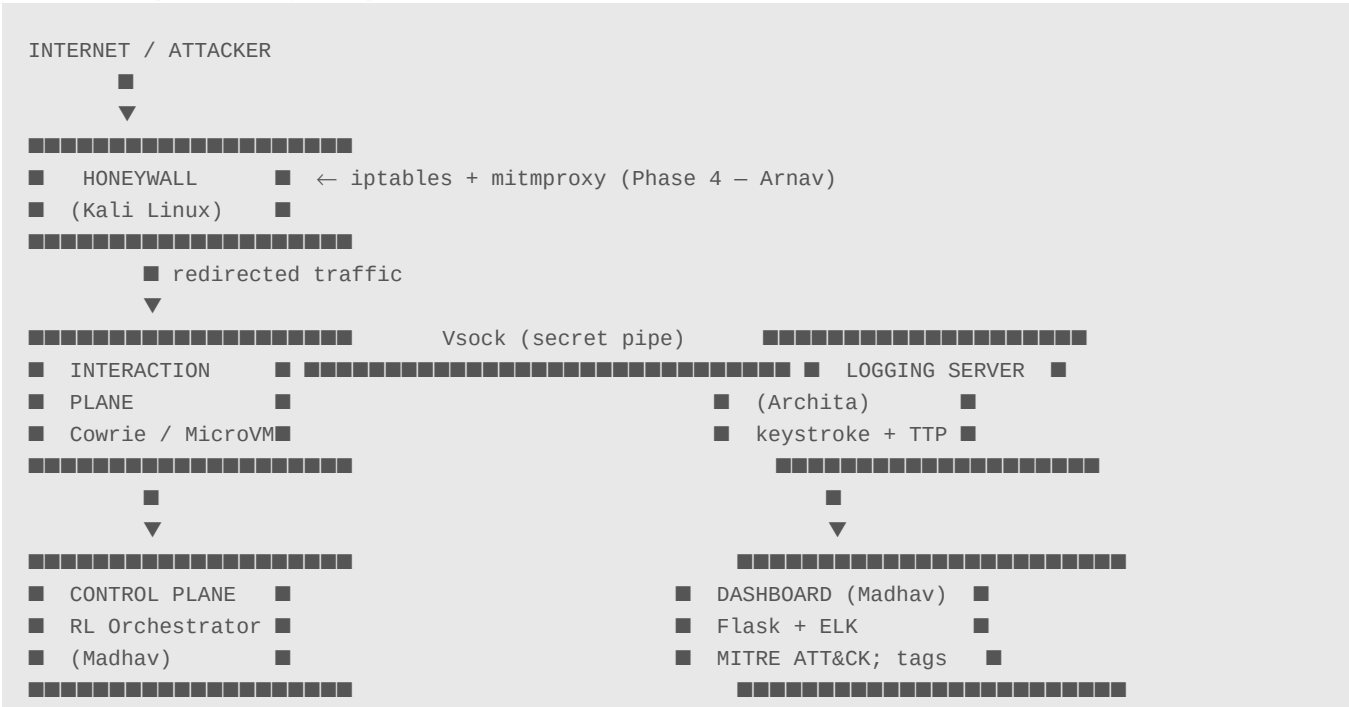
Layer	Technology	Purpose
Virtualisation	Firecracker MicroVM + KVM	Isolated, lightweight sandboxes (125 ms boot, 5 MiB RAM)
Emulation	Cowrie SSH/Telnet Honeypot	Low-interaction service emulation
Behaviour Synthesis	OpenAI / Ollama LLM	Human-like terminal responses
Data Generation	Python Faker + Shell scripts	Realistic fake users, logs, history
Logging	Linux Vsock + Python asyncio	Out-of-band keystroke & payload capture
Containment	iptables + mitmproxy	Honeywall preventing real outbound harm
Intelligence	MITRE ATT&CK + ELK Stack	TTP mapping and real-time dashboard
Orchestration	Python + Flask + Systemd	Single control plane tying all phases together

2. Architecture Summary

AADE is divided into three logical planes, each mapping to one or more implementation phases:

Plane	Components	Owner
Interaction Plane	Cowrie low-interaction containers, Firecracker MicroVM high-interaction containers, honeypots (Vuln)	Shirvardkar (Arnav)
Control Plane	Reinforcement-Learning orchestrator, SDN session migration (Madhav), host-based decoy lifecycle re	Madhav (Ch)
Intelligence Plane	Vsock out-of-band logging, payload capture, automatic TTP Applying MITRE ATT&CK (Sashidhar)	Archita (Sashidhar)

Data Flow (text diagram)



3. Team Responsibilities Matrix

Task / Deliverable	Archita Roy	Sharvari Achrekar	Arnav Shirvadkar	Madhav Shukla
VMware nested-virt setup	–	–	PRIMARY	–
Firecracker binary install	–	–	PRIMARY	SUPPORT
KVM kernel module config	–	–	PRIMARY	–
Fake filesystem build	–	PRIMARY	–	–
Bash history / log noise	–	PRIMARY	SUPPORT	–
Faker dummy-data scripts	–	PRIMARY	–	–
Vsock kernel config	–	–	SUPPORT	–
Python Vsock listener	PRIMARY	–	SUPPORT	–
Out-of-band log storage	PRIMARY	–	–	SUPPORT
iptables honeywall rules	–	–	PRIMARY	–
mitmproxy C2 sinkhole	–	–	PRIMARY	SUPPORT
Master orchestrator script	–	–	–	PRIMARY
Flask dashboard	–	–	–	PRIMARY
MITRE ATT&CK tagging	PRIMARY	–	–	SUPPORT
Integration & end-to-end tests	SUPPORT	SUPPORT	SUPPORT	PRIMARY
Documentation & report	ALL	ALL	ALL	ALL

Legend: *PRIMARY* = responsible owner; *SUPPORT* = assists; – = not involved in this task.

PHASE 1 — Environment & Virtualisation · Owner: Arnav Shirvadkar

4. Phase 1 — Environment & Virtualisation

Arnav is responsible for preparing the entire hardware and virtualisation layer. This phase is the foundation — nothing else works until KVM and Firecracker are running. Target host OS: **Kali Linux 2024.x** (bare-metal or VMware with nested virtualisation enabled).

4.1 Pre-flight Checklist

Check	Command	Expected Output
CPU virtualisation flag	<code>grep -E 'vmx svm' /proc/cpuinfo head -1</code>	vmx (Intel) or svm (AMD) present
KVM module loaded	<code>lsmod grep kvm</code>	kvm_intel or kvm_amd
KVM device exists	<code>ls -la /dev/kvm</code>	/dev/kvm crw-rw---- exists
Kernel version >= 5.10	<code>uname -r</code>	e.g. 6.x.x-kali
Available disk space	<code>df -h /</code>	At least 20 GB free
RAM	<code>free -h</code>	At least 8 GB total

4.2 Step-by-Step Setup

Step 1 — Enable nested virtualisation in VMware (if running Kali inside VMware)

Power off the VM. Edit the .vmx file and add:

```
vhv.enable = TRUE cpuid.1.ecx = "-----:-----:-----:-----:-----:-----:--H:-----"
```

Then power on and verify with the `cpuinfo grep` command above.

Step 2 — Install KVM and virtualisation tools

```
sudo apt update && sudo apt upgrade -y sudo apt install -y qemu-kvm libvirt-daemon-system virtinst
bridge-utils cpu-checker sudo kvm-ok # must print: KVM acceleration can be used sudo usermod -aG
kvm,libvirt $USER newgrp kvm
```

Step 3 — Download Firecracker binaries

Firecracker is a single static binary. Download the latest release from GitHub:

```
RELEASE=v1.7.0 arch=$(uname -m) curl -LO https://github.com/firecracker-microvm/firecracker/releases/d
ownload/${RELEASE}/firecracker-${RELEASE}-${arch}.tgz tar -xzf firecracker-${RELEASE}-${arch}.tgz sudo
mv release-${RELEASE}-${arch}/firecracker-${RELEASE}-${arch} /usr/local/bin/firecracker sudo mv
release-${RELEASE}-${arch}/jailer-${RELEASE}-${arch} /usr/local/bin/jailer sudo chmod +x
/usr/local/bin/firecracker /usr/local/bin/jailer firecracker --version # verify
```

Step 4 — Download a kernel for Firecracker MicroVMs

Firecracker requires a stripped kernel image (vmlinux format):

```
mkdir -p ~/aade/kernels curl -fsSL
https://s3.amazonaws.com/spec.ccfc.min/img/quickstart_guide/x86_64/kernels/vmlinux.bin \ -o
~/aade/kernels/vmlinux.bin
```

Step 5 — Verify the Firecracker socket API

```
# In terminal 1 — start firecracker rm -f /tmp/fc.sock firecracker --api-sock /tmp/fc.sock # In
terminal 2 — query the API curl --unix-socket /tmp/fc.sock http://localhost/ # Expected:
{"id":"anonymous-instance","state":"Not started","vmm_version":"..."}
```

4.3 GitHub Sources & Downloads for Phase 1

Resource	URL	What to clone / download
Firecracker	https://github.com/firecracker-microvm/firecracker	Binary releases — see Step 3
Firecracker Go SDK	https://github.com/firecracker-microvm/firecracker-go-sdk	Optional: programmatic VM management
Firecracker Python SDK	https://github.com/iann0036/pycracker	Lightweight Python wrapper for API
Getting-Started rootfs	https://github.com/firecracker-microvm/firecracker/blob/main/docs/getting-started.md	Official tutorial
KVM Forum scripts	https://github.com/kvmtool/kvmtool	Reference kernel build scripts
QEMU utilities	<code>apt install qemu-utils</code>	For converting disk images (<code>qemu-img</code>)

pip / apt packages for Phase 1

Package	Install Command	Purpose
firecracker binary	See curl command above	The MicroVM engine
qemu-kvm	<code>sudo apt install qemu-kvm</code>	Hardware virtualisation
libvirt-daemon	<code>sudo apt install libvirt-daemon-system</code>	VM lifecycle management
python3-requests	<code>pip install requests</code>	Call Firecracker REST API from Python
boto3 (optional)	<code>pip install boto3</code>	If using AWS Firecracker AMIs

PHASE 2 — Gold Image Creation · Owner: Sharvari Achrekar

5. Phase 2 — Gold Image Creation

Sharvari builds the *fake operating system* that the attacker will see the moment they enter the honeypot. The goal is to create a disk image (ext4 filesystem) that looks like a real developer's Ubuntu workstation — with aged log files, plausible bash history, fake SSH keys, dummy project folders, and fabricated user accounts. This is the primary mechanism that fools human attackers and prevents easy fingerprinting.

5.1 Building the Fake Filesystem

Step 1 — Create the root filesystem image

```
# Create a 1 GB sparse ext4 image dd if=/dev/zero of=~/aade/images/rootfs.ext4 bs=1M count=1024
mkfs.ext4 ~/aade/images/rootfs.ext4 mkdir -p /mnt/gold sudo mount ~/aade/images/rootfs.ext4 /mnt/gold
```

Step 2 — Bootstrap a minimal Debian/Ubuntu base into the image

```
sudo apt install -y debootstrap sudo debootstrap --arch=amd64 focal /mnt/gold
http://archive.ubuntu.com/ubuntu/ # This takes ~5 minutes. Result is a full Ubuntu 20.04 minimal root.
```

Step 3 — Add a fake user account

```
sudo chroot /mnt/gold /bin/bash << 'EOF' useradd -m -s /bin/bash devuser echo "devuser:password123" |
chpasswd usermod -aG sudo devuser EOF
```

5.2 Human-Noise Scripts (Python Faker)

Sharvari writes a Python script using the **Faker** library to populate the filesystem with realistic artefacts before the image is sealed.

File: generate_noise.py

```
#!/usr/bin/env python3 """Populate /mnt/gold with human-like artefacts.""" import os, random,
subprocess from faker import Faker from datetime import datetime, timedelta import pathlib ROOT =
"/mnt/gold/home/devuser" fake = Faker() # 1. Fake bash history history_cmds = [ "ls -la", "cd
/var/www/html", "git pull origin main", "python3 manage.py runserver", "sudo apt update", "ssh
admin@192.168.1.45", "cat /etc/passwd", "nano config.py", "curl http://api.internal/health", "docker
ps", "mysql -u root -p", "grep -r 'password' .", "tail -f /var/log/nginx/access.log", "ps aux | grep
python", "kill -9 1234", "tar -czf backup_$(date +%F).tar.gz /opt/app", ] with
open(f"{ROOT}/.bash_history", "w") as f: for _ in range(200): f.write(random.choice(history_cmds) +
"\n") # 2. Fake SSH keys ssh_dir = f"{ROOT}/.ssh" os.makedirs(ssh_dir, exist_ok=True) with
open(f"{ssh_dir}/known_hosts", "w") as f: for _ in range(10): f.write(f"{fake.ipv4()} ssh-rsa
AAAAB3NzaC{fake.md5()}\n") # 3. Salary / HR documents (high-value bait) docs_dir = f"{ROOT}/Documents"
os.makedirs(docs_dir, exist_ok=True) bait_files = ["Salary_2026.xlsx", "Q3_Financial_Report.pdf",
"employee_database_backup.sql", "vpn_credentials.txt"] for fname in bait_files: with
open(f"{docs_dir}/{fname}", "w") as f: f.write(fake.text(max_nb_chars=2000)) # 4. Fake project
directory proj = f"{ROOT}/projects/webapp" os.makedirs(proj, exist_ok=True) with
open(f"{proj}/config.py", "w") as f: f.write(f"DB_PASSWORD = '{fake.password()}'\n")
f.write(f"SECRET_KEY = '{fake.sha256()}'\n") f.write(f"API_TOKEN = '{fake.uuid4()}'\n") # 5. System
logs with realistic timestamps log_dir = "/mnt/gold/var/log" os.makedirs(log_dir, exist_ok=True) with
open(f"{log_dir}/syslog", "w") as f: now = datetime.now() for i in range(500): ts = (now -
timedelta(minutes=i*3)).strftime("%b %d %H:%M:%S") f.write(f"{ts} devbox systemd[1]:
{fake.sentence()}\n") print("[+] Gold image noise generation complete.")
```

Step 4 — Seal the image (unmount and compress)

```
sudo umount /mnt/gold # Create a snapshot for rapid re-cloning cp ~/aade/images/rootfs.ext4
~/aade/images/rootfs_gold.ext4 e2fsck -f ~/aade/images/rootfs_gold.ext4 # check filesystem integrity
```

5.3 GitHub Sources & Downloads for Phase 2

Resource	URL	Purpose
Faker (Python)	https://github.com/joke2k/faker	Generate realistic dummy data
Cowrie honeypot	https://github.com/cowrie/cowrie	Low-interaction SSH/Telnet honeypot
Debootstrap	apt install debootstrap	Bootstrap minimal Ubuntu filesystem
OpenSnitch (optional)	https://github.com/evilsocket/opensnitch	Monitor outbound connections inside image
Honeyfs	https://github.com/micheloosterhof/cowrie	Cowrie's fake filesystem helper
Age file timestamps	https://github.com/manuelcortez/easyTimestamp	Script to back-date file timestamps

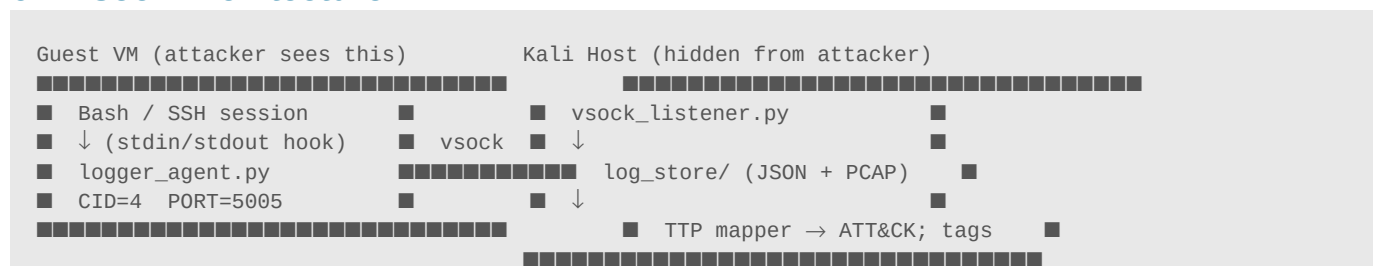
Package	Install Command	Purpose
faker	pip install faker	Fake user data generation
selenium	pip install selenium	Simulate browser history
pyautogui	pip install pyautogui	Simulate desktop activity
debootstrap	sudo apt install debootstrap	Build rootfs inside loop device
e2fsprogs	sudo apt install e2fsprogs	Filesystem tools (mkfs.ext4, e2fsck)

PHASE 3 — Stealth Vsock Logging · Owner: Archita Roy

6. Phase 3 — Stealth Vsock Logging (Intelligence Plane)

Archita builds the *secret pipe* — a Linux **vsock** (virtual socket) channel that runs between the MicroVM guest and the Kali host entirely inside the hypervisor. The attacker has no visibility into this channel and cannot delete or tamper with the logs even if they gain root on the guest. Every keystroke, every command output, and every file downloaded is streamed in real-time to the host.

6.1 Vsock Architecture



6.2 Python Vsock Scripts

Host-side listener: vsock_listener.py

```
#!/usr/bin/env python3 """Run on Kali host. Receives all keystrokes from guest over vsock.""" import
socket, json, os, datetime, asyncio VSOCK_PORT = 5005 LOG_DIR = os.path.expanduser("~/aade/logs")
os.makedirs(LOG_DIR, exist_ok=True) def save_event(event: dict): ts =
datetime.datetime.utcnow().isoformat() day = datetime.date.today().isoformat() path =
os.path.join(LOG_DIR, f"session_{day}.jsonl") event["timestamp"] = ts with open(path, "a") as f:
f.write(json.dumps(event) + "\n") # Print live to terminal print(f"[{ts}] CMD: {event.get('cmd','')}")
def main(): sock = socket.socket(socket.AF_VSOCK, socket.SOCK_STREAM)
sock.bind((socket.VMADDR_CID_HOST, VSOCK_PORT)) sock.listen(5) print(f"[*] vsock listener on port
{VSOCK_PORT}") while True: conn, addr = sock.accept() print(f"[+] Connection from CID {addr[0]}") buf
= b"" while True: data = conn.recv(4096) if not data: break buf += data while b"\n" in buf: line, buf =
buf.split(b"\n", 1) try: event = json.loads(line.decode()) save_event(event) except
json.JSONDecodeError: pass conn.close() if __name__ == "__main__": main()
```

Guest-side agent: `logger_agent.py` (runs inside MicroVM)

```
#!/usr/bin/env python3 """Inject into guest .bashrc. Sends every command to host via vsock.""" import
socket, json, os, subprocess, sys HOST_CID = 2 # Firecracker host CID is always 2 VSOCK_PORT = 5005 def
send_event(event: dict): try: s = socket.socket(socket.AF_VSOCK, socket.SOCK_STREAM) s.settimeout(2)
s.connect((HOST_CID, VSOCK_PORT)) s.sendall((json.dumps(event) + "\n").encode()) s.close() except
Exception: pass # silent failure so attacker notices nothing # Hook into bash via PROMPT_COMMAND # Add
to guest .bashrc: # export PROMPT_COMMAND='python3 /usr/local/bin/logger_agent.py "$?" "${history 1}"'
if __name__ == "__main__": exit_code = sys.argv[1] if len(sys.argv) > 1 else "0" cmd = sys.argv[2] if
len(sys.argv) > 2 else "" send_event({"cmd": cmd.strip(), "exit": exit_code, "user":
os.getenv("USER", "?"), "cwd": os.getcwd(), "host": os.uname().nodename})
```

Enable vsock in Firecracker VM config (JSON)

```
{ "vsock": { "guest_cid": 4, "uds_path": "/tmp/vsock.sock" } } # Send this to Firecracker API: curl
--unix-socket /tmp/fc.sock -X PUT http://localhost/vsock \ -H "Content-Type: application/json" \ -d
'{"guest_cid": 4, "uds_path": "/tmp/vsock.sock"}
```

MITRE ATT&CK TTP Tagger

```
# tcp_mapper.py - maps captured commands to ATT&CK; techniques TTP_MAP = { "wget|curl": "T1105 Ingress  
Tool Transfer", "chmod.*x": "T1222 File and Directory Permissions Modification", "cat /etc/passwd":  
"T1003 OS Credential Dumping", "ssh ": "T1021.004 Remote Services: SSH", "python.*-c": "T1059.006  
Command and Scripting Interpreter: Python", "nc |netcat": "T1059 Command and Scripting Interpreter",
```

```
"crontab": "T1053.003 Scheduled Task/Job: Cron", "useradd|adduser": "T1136 Create Account", } import
re, json def tag_event(event: dict) -> dict: cmd = event.get("cmd", "") for pattern, ttp in
TTP_MAP.items(): if re.search(pattern, cmd): event.setdefault("ttps", []).append(ttp) return event
```

6.3 GitHub Sources & Downloads for Phase 3

Resource	URL	Purpose
Cowrie (keystroke logging)	https://github.com/cowrie/cowrie	Full-featured SSH honeypot with built-in logging
MITRE ATT&CK Python	https://github.com/mitre-attack/mitreattack-python	Query ATT&CK knowledge base
ELK Stack (Elasticsearch)	https://github.com/elastic/elasticsearch	Store and search log events
Logstash	https://github.com/elastic/logstash	Ingest pipeline from JSONL logs
Kibana	https://github.com/elastic/kibana	Visual dashboard for ATT&CK events
Vsock docs	https://www.kernel.org/doc/html/latest/networking/kvsock.html	Kernel docs for AF_VSOCK

Package	Install Command	Purpose
mitreattack-python	<code>pip install mitreattack-python</code>	TTP lookup from captured commands
elasticsearch	<code>pip install elasticsearch</code>	Python client for ELK
scapy	<code>pip install scapy</code>	Deep packet capture analysis
watchdog	<code>pip install watchdog</code>	Filesystem event triggers in guest
psutil	<code>pip install psutil</code>	Resource noise injection (CPU/RAM)

PHASE 4 — Honeywall & Containment · Owner: Arnav Shirvadkar

7. Phase 4 — Honeywall & Containment

The honeywall ensures that malware dropped by the attacker cannot escape and harm the real internet (no DDoS, no spam, no ransomware propagation). It also intercepts C2 (command-and-control) traffic, feeds the attacker fake success responses, and lets Archita's logging plane see everything.

7.1 iptables Rules (honeywall_setup.sh)

```
#!/bin/bash # honeywall_setup.sh – Run as root on Kali host # Assumptions: # eth0 = external NIC (real internet) # virbr0 = bridge to honeypot VMs (192.168.100.0/24) HONEYPOT_NET="192.168.100.0/24" KALI_IP="192.168.100.1" # 1. Flush existing rules iptables -F; iptables -X; iptables -t nat -F; iptables -t nat -X # 2. Default DROP on FORWARD chain (honeypots cannot talk to each other freely) iptables -P FORWARD DROP # 3. Allow ESTABLISHED/RELATED (return traffic for attacker sessions) iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT # 4. Allow inbound to honeypots (attacker can reach them) iptables -A FORWARD -d $HONEYPOT_NET -j ACCEPT # 5. BLOCK all outbound from honeypots to real internet iptables -A FORWARD -s $HONEYPOT_NET ! -d $HONEYPOT_NET -j DROP # 6. Redirect honeypot DNS to local sinkhole iptables -t nat -A PREROUTING -s $HONEYPOT_NET -p udp --dport 53 -j DNAT --to $KALI_IP:5353 # 7. Redirect honeypot HTTP/HTTPS to mitmproxy for logging iptables -t nat -A PREROUTING -s $HONEYPOT_NET -p tcp --dport 80 -j DNAT --to $KALI_IP:8080 iptables -t nat -A PREROUTING -s $HONEYPOT_NET -p tcp --dport 443 -j DNAT --to $KALI_IP:8443 # 8. Log all outbound attempts before dropping iptables -A FORWARD -s $HONEYPOT_NET -j LOG --log-prefix "[HONEYPOT-OUTBOUND]" --log-level 4 echo "[+] Honeywall active." iptables -L -n -v
```

7.2 mitmproxy C2 Sinkhole Setup

mitmproxy captures all HTTP/HTTPS traffic from the honeypot and feeds fake 200 OK responses so the attacker's C2 check-ins appear successful while we record every URL and payload.

```
# Install mitmproxy pip install mitmproxy # c2_sinkhole.py – mitmproxy addon script from mitmproxy import http import json, datetime LOG_FILE = "/root/aade/logs/c2_traffic.jsonl" class C2Sinkhole: def request(self, flow: http.HTTPFlow): # Log the C2 request entry = { "ts": datetime.datetime.utcnow().isoformat(), "method": flow.request.method, "url": flow.request.pretty_url, "headers": dict(flow.request.headers), "body": flow.request.get_text(strict=False), } with open(LOG_FILE, "a") as f: f.write(json.dumps(entry) + "\n") # Return fake 200 OK to fool the attacker flow.response = http.Response.make( 200, b'{"status":"ok","message":"beacon received"}', {"Content-Type": "application/json"} ) addons = [C2Sinkhole()] # Launch: # mitmproxy --listen-host 0.0.0.0 --listen-port 8080 -s c2_sinkhole.py --ssl-insecure
```

DNS Sinkhole (dnsmasq)

```
# /etc/dnsmasq.d/honeypot-sinkhole.conf # Redirect all honeypot DNS to our fake server address=192.168.100.1 # return our IP for every domain # Start dnsmasq on port 5353 (non-privileged) dnsmasq --no-daemon --listen-address=192.168.100.1 --port=5353 \ --conf-file=/etc/dnsmasq.d/honeypot-sinkhole.conf
```

Resource	URL	Purpose
mitmproxy	https://github.com/mitmproxy/mitmproxy	Intercept & forge HTTP/S responses
dnsmasq	apt install dnsmasq	DNS sinkhole
iptables-persistent	apt install iptables-persistent	Survive reboots
tcpdump	apt install tcpdump	PCAP capture for forensics
Wireshark	apt install wireshark	Analyse captured traffic

PHASE 5 — Orchestrator & Dashboard · Owner: Madhav Shukla

8. Phase 5 — Orchestrator & Dashboard

Madhav writes the glue that ties everything together: a master orchestrator that launches VMs, monitors sessions, and migrates attackers from low-interaction (Cowrie) to high-interaction (MicroVM) when they look sufficiently sophisticated. He also builds the Flask-based real-time dashboard showing live ATT&CK; tags, active sessions, and alert counts.

8.1 Master Orchestrator (orchestrator.py)

```
#!/usr/bin/env python3 """AADE Master Orchestrator – ties all phases together.""" import subprocess,
threading, time, json, os, requests, logging logging.basicConfig(level=logging.INFO,
format='%(%asctime)s [ORCH] %(message)s') log = logging.getLogger(__name__) FC_SOCKET = "/tmp/fc.sock"
KERNEL = os.path.expanduser("~/aade/kernels/vmlinux.bin") ROOTFS =
os.path.expanduser("~/aade/images/rootfs_gold.ext4") LOG_DIR = os.path.expanduser("~/aade/logs")
VM_CONFIG = { "boot-source": {"kernel_image_path": KERNEL, "boot_args": "console=ttyS0 reboot=k
panic=1 pci=off"}, "drives": [{"drive_id": "rootfs", "path_on_host": ROOTFS, "is_root_device": True,
"is_read_only": False}], "machine-config": {"vcpu_count": 2, "mem_size_mib": 512}, "vsock":
{"guest_cid": 4, "uds_path": "/tmp/vsock.sock"}, "network-interfaces":
[{"iface_id": "eth0", "guest_mac": "AA:BB:CC:DD:EE:FF", "host_dev_name": "tap0"}], } def fc_put(path: str,
data: dict): r = requests.put(f"http://localhost{path}", headers={"Content-Type": "application/json"},
data=json.dumps(data), timeout=5, # use unix socket via requests adapter (see urllib3 workaround) )
log.info(f"FC PUT {path} → {r.status_code}") return r def start_microvm(): """Launch Firecracker and
configure the VM via REST API.""" os.system(f"rm -f {FC_SOCKET}") proc =
subprocess.Popen(["firecracker", "--api-sock", FC_SOCKET], stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL) time.sleep(0.5) # wait for socket for path, data in VM_CONFIG.items(): if
path != "network-interfaces": fc_put(f"/{path}", data) for nic in VM_CONFIG["network-interfaces"]:
fc_put(f"/network-interfaces/{nic['iface_id']}", nic) fc_put("/actions",
{"action_type": "InstanceStart"}) log.info("MicroVM started.") return proc def
watch_cowrie_log(cowrie_json_log: str): """Tail Cowrie's JSON log. Migrate session to MicroVM if
attacker is advanced.""" import tailer for line in tailer.follow(open(cowrie_json_log)): try: ev =
json.loads(line) if ev.get("eventid") == "cowrie.command.input": cmd = ev.get("input", "") if any(k in
cmd for k in ["wget", "curl", "nc ", "python", "perl", "bash -i"]): log.warning(f"Advanced attacker
detected! CMD: {cmd}") # TODO: trigger SDN migration to MicroVM except json.JSONDecodeError: pass if
__name__ == "__main__": vm_proc = start_microvm() t = threading.Thread(target=watch_cowrie_log,
args=["/var/log/cowrie/cowrie.json"], daemon=True) t.start() log.info("Orchestrator running. Press
Ctrl-C to stop.") try: vm_proc.wait() except KeyboardInterrupt: vm_proc.terminate()
```

8.2 Flask Dashboard (dashboard.py)

```
from flask import Flask, render_template_string, jsonify import json, os, glob from collections import
Counter app = Flask(__name__) LOG_DIR = os.path.expanduser('~/.aade/logs') # TEMPLATE is a Jinja2 HTML
string (see full source in dashboard.py) # Key routes: def load_events(): events = [] for f in
sorted(glob.glob(f'{LOG_DIR}/*.jsonl'))[-3:]: with open(f) as fh: for line in fh: try:
events.append(json.loads(line)) except: pass return list(reversed(events))[-200:] @app.route('/') def
index(): events = load_events() ttp_set = set(t for e in events for t in e.get('ttps', [])) return
render_template_string(TEMPLATE, events=events[-50:], total=len(events), sessions=1,
ttps=len(ttp_set)) @app.route('/api/events') def api_events(): return jsonify(load_events()[-100:]) if
__name__ == '__main__': app.run(host='0.0.0.0', port=5000, debug=False)
```

8.3 GitHub Sources & Downloads for Phase 5

Resource	URL	Purpose
Cowrie honeypot	https://github.com/cowrie/cowrie	Low-interaction SSH used by orchestrator
Flask	https://github.com/pallets/flask	Web framework for dashboard

Tailer (log following)	pip install tailer	Follow growing log files in Python
Stable-baselines3 (RL)	https://github.com/DLR-RM/stable-baselines3	RL orchestrator for session escalation
Socket.IO (live updates)	https://github.com/miguelgrinberg/Flask-SocketIO	Push live events to dashboard browser
Chart.js	https://github.com/chartjs/Chart.js	Attack timeline charts in dashboard
Bootstrap 5	https://github.com/twbs/bootstrap	Dashboard CSS framework (CDN OK)

Package	Install Command	Purpose
flask	pip install flask	Dashboard web server
flask-socketio	pip install flask-socketio	Live event push to browser
tailer	pip install tailer	Tail log files
stable-baselines3	pip install stable-baselines3	Reinforcement learning orchestrator
twisted	pip install twisted	Protocol emulation framework
paramiko	pip install paramiko	SSH management for guest control

9. Integration & End-to-End Testing

Integration is owned by Madhav with all four members contributing. Run these tests in order — each depends on the previous phase being complete.

#	Test	Command / Method	Pass Condition	Owner
T1	KVM available	kvm-ok	'KVM acceleration can be used'	Arnav
T2	Firecracker binary	firecracker --version	Version string printed	Arnav
T3	MicroVM boots	Start VM via API, wait 2 s, check /action	State = Running	Arnav
T4	Gold image mounts	mount rootfs.ext4 /mnt/test && ls /mnt/test/home	testuser dir present	Sharvari
T5	Fake files present	ls /mnt/test/home/devuser/Documents	Salary_2026.xlsx exists	Sharvari
T6	Vsock listener up	python3 vsock_listener.py &	'vsock listener on port 5005'	Archita
T7	Guest agent sends	Inside VM: python3 logger_agent.py 0	'T1003' appears in log file	Archita
T8	Honeywall blocks	From VM: ping 8.8.8.8	Request times out	Arnav
T9	mitmproxy captures	From VM: curl http://evil.com	200 OK returned + entry in c2_traffic.jsonl	Arnav
T10	Dashboard live	Open http://localhost:5000	Events visible in browser	Madhav
T11	ATT&CK tagging	Run 'cat /etc/passwd' inside VM	T1003 tag appears in dashboard	Archita
T12	Full attack sim	Run metasploit meterpreter from separate VM	Att VM uses log correctly	ALL

10. Full Dependency & Install Reference

Run the following master install script on the Kali host once before starting Phase 1:

```
#!/bin/bash # master_install.sh – Run once on Kali Linux host set -e echo "[+] System packages" sudo
apt update && sudo apt install -y \ qemu-kvm libvirt-daemon-system virtinst bridge-utils cpu-checker \
dnsmasq iptables-persistent tcpdump wireshark tshark debootstrap \ python3-pip python3-venv git curl
wget jq net-tools iproute2 \ e2fsprogs squashfs-tools echo "[+] Python packages" pip3 install --user \
faker selenium pyautogui \ scapy twisted paramiko psutil watchdog \ mitmproxy requests flask
flask-socketio tailer \ elasticsearch mitreattack-python stable-baselines3 \ gymnasium torch echo "[+]
Firecracker" RELEASE=v1.7.0; ARCH=$(uname -m) curl -LO https://github.com/firecracker-microvm/firecrac
ker/releases/download/${RELEASE}/firecracker-${RELEASE}-${ARCH}.tgz tar -xzf
firecracker-${RELEASE}-${ARCH}.tgz sudo mv release-${RELEASE}-${ARCH}/firecracker-${RELEASE}-${ARCH}
/usr/local/bin/firecracker sudo mv release-${RELEASE}-${ARCH}/jailer-${RELEASE}-${ARCH}
/usr/local/bin/jailer sudo chmod +x /usr/local/bin/firecracker /usr/local/bin/jailer echo "[+] Cowrie
honeypot" git clone https://github.com/cowrie/cowrie ~/aade/cowrie cd ~/aade/cowrie pip3 install
--user -r requirements.txt cp etc/cowrie.cfg.dist etc/cowrie.cfg echo "[+] Kernel image for
Firecracker" mkdir -p ~/aade/kernels curl -fsSL
https://s3.amazonaws.com/spec.ccfc.min/img/quickstart_guide/x86_64/kernels/vmlinux.bin \ -o
~/aade/kernels/vmlinux.bin echo "[✓] All dependencies installed."
```

Full Python Dependency List (pip freeze reference)

Package	Version (min)	Used In
faker	>=20.0	Phase 2 — Gold Image
selenium	>=4.0	Phase 2 — Browser history simulation
scapy	>=2.5	Phase 3 — Packet capture
twisted	>=23.0	Phase 1/3 — Protocol emulation
paramiko	>=3.0	Phase 1/5 — SSH management
psutil	>=5.9	Phase 2/3 — Resource noise
watchdog	>=3.0	Phase 3 — Filesystem triggers
mitmproxy	>=10.0	Phase 4 — C2 sinkholing
requests	>=2.31	Phase 5 — Firecracker API
flask	>=3.0	Phase 5 — Dashboard
flask-socketio	>=5.0	Phase 5 — Live push events
tailer	>=0.4	Phase 5 — Log file tailing
elasticsearch	>=8.0	Phase 3 — Log storage (optional ELK)
mitreattack-python	>=2.0	Phase 3 — ATT&CK tagging
stable-baselines3	>=2.0	Phase 5 — RL orchestrator
gymnasium	>=0.29	Phase 5 — RL environment
torch	>=2.0	Phase 5 — RL neural network backend

11. Risk Register & Mitigations

Risk	Severity	Probability	Mitigation	Owner
Anti-VM detection (CPUID timing)	HIGH	MEDIUM	Use CPU passthrough (--cpuid-override); randomise CPU jitter; use bare metal	Arnav
Attacker escapes MicroVM (cgroup breackout)	HIGH	LOW	KVM hardware isolation; jailer sandbox; vsock is read-only from guest side	Arnav
Legal liability (captured malware spreads)	MEDIUM	LOW	iptables DROP all outbound; mitmproxy; Antix intercepts; never expose honeypot	Arnav
Gold image looks fake (missing entropy)	MEDIUM	MEDIUM	Run Faker + Selenium pre-aging; use real kernel, not emulated; vary file hashes	Arnav
Vsock logging can be silently disabled	MEDIUM	LOW	Logger agent runs as root; PROMPT_COMMAND is in system-wide /etc/profile	Arnav
Dashboard leaks live IPs to attacker	HIGH	LOW	Dashboard binds to 127.0.0.1 only; never expose port 5000 on honeywall	Arnav
ELK stack performance under high volume	MEDIUM	MEDIUM	Rate-limit at Logstash ingest; use JSON format files; Masfallo back; disk quota	Arnav
Python PROMPT_COMMAND hook killed by attacker	LOW	MEDIUM	Backup: Cowrie captures all commands independently; vsock agent is airdrop	Arnav

12. Project Timeline

Suggested 8-week sprint plan (adjust dates to your college calendar):

Week	Task	Owner	Deliverable
1	Setup Kali, enable nested-virt, install KVM	Arnav	KVM working, kvm-ok passes
1	Research Firecracker docs & quickstart	Arnav	Firecracker binary installed
2	Download & boot first MicroVM	Arnav	VM running, can SSH in
2	Firecracker API Python wrapper	Arnav + Madhav	fc_api.py script
3	Debootstrap Ubuntu rootfs	Sharvari	rootfs.ext4 with devuser
3	Faker noise generation script	Sharvari	generate_noise.py working
3	Cowrie install & SSH test	Sharvari	Cowrie logs first SSH login
4	Vsock kernel config in MicroVM	Archita + Arnav	/dev/vsock present in guest
4	Host vsock_listener.py	Archita	Events appear in log file
4	Guest logger_agent.py	Archita	Commands streamed to host
5	iptables honeywall rules	Arnav	Ping test from VM fails
5	mitmproxy C2 sinkhole	Arnav	c2_traffic.jsonl populated
5	DNS sinkhole (dnsmasq)	Arnav	All DNS returns 192.168.100.1
6	TTP mapper (ttp_mapper.py)	Archita	ATT&CK tags in log entries
6	Master orchestrator.py	Madhav	Single script starts all phases
7	Flask dashboard	Madhav	Events visible at localhost:5000
7	Integration testing T1–T11	ALL	All tests pass
8	Full attack simulation (Metasploit)	ALL	T12 passes, final demo ready
8	Documentation & presentation polish	ALL	Final report + slide deck

13. Glossary

Term	Definition
AADE	Autonomous Adaptive Deception Environment — the name of this project.
ATT&CK	MITRE's framework of adversary Tactics, Techniques, and Procedures (TTPs).
C2	Command and Control — attacker server that malware phones home to.
Cowrie	Open-source medium-interaction SSH/Telnet honeypot written in Python.
CID	Context ID — unique identifier for a VM in the vsock addressing scheme.
Debootstrap	Tool to install a Debian/Ubuntu base system into a directory/image.
ext4	Linux fourth extended filesystem — standard Linux disk format.
Faker	Python library to generate realistic fake personal and system data.
Firecracker	AWS open-source MicroVM hypervisor used by Lambda and Fargate.
Gold Image	A pre-configured, human-noise-filled disk snapshot ready to clone.
Honeynet	A network of multiple honeypots sharing a common monitoring plane.
Honeytoken	Fake credential or file designed to trigger an alert if accessed.
Honeywall	Network gateway that controls traffic to/from honeypots.
IDS	Intrusion Detection System — passive monitor that alerts on suspicious traffic.
IPS	Intrusion Prevention System — active monitor that can block traffic.
KVM	Kernel-based Virtual Machine — Linux hypervisor using CPU virtualisation.
LLM	Large Language Model (e.g. GPT-4, Llama 3) used to generate realistic terminal responses.
mitmproxy	Open-source interactive HTTPS proxy used for C2 sinkholing.
MicroVM	Extremely lightweight virtual machine (~125 ms boot, 5 MiB RAM) via Firecracker.
RL	Reinforcement Learning — AI technique where an agent learns by reward/penalty.
SDN	Software-Defined Networking — network behaviour controlled by software logic.
TTP	Tactic, Technique, Procedure — a specific attacker behaviour categorised by MITRE.
Vsock	Linux AF_VSOCK socket family — hypervisor-level communication channel between guest and host.
Zero-day	A vulnerability unknown to the vendor and therefore unpatched.

Project AADE — Quick Reference Card

Key Repositories to Clone

`https://github.com/firecracker-microvm/firecracker`

`https://github.com/cowrie/cowrie`

`https://github.com/joke2k/faker`

`https://github.com/mitmproxy/mitmproxy`

`https://github.com/mitre-attack/mitreattack-python`

`https://github.com/pallets/flask`

`https://github.com/DLR-RM/stable-baselines3`

One-liner to clone all repos

```
mkdir -p ~/aade && cd ~/aade git clone https://github.com/firecracker-microvm/firecracker git
clone https://github.com/cowrie/cowrie git clone https://github.com/mitmproxy/mitmproxy git
clone https://github.com/mitre-attack/mitreattack-python git clone
https://github.com/DLR-RM/stable-baselines3
```

In modern defence, deception is not optional — it is strategic superiority.